Módulo Profesional 0616

PROYECTO DE DESARROLLO DE APLICACIONES WEB

Documento para la Especificación de Casos de Prueba

CICLO FORMATIVO DE GRADO SUPERIOR EN

DESARROLLO DE APLICACIONES WEB

MODALIDAD ONLINE

Grupo 8 DAW

UBIKATE

Especificación de Caso de Prueba

API, usuarios y datos

Historial de Revisiones

Fecha	Versión	Descripción	Autor
02/11/2026	1.0	Primera ejecución. Siete casos sobre el backend de UBIKATE.	Manuel Rodríguez Vaca

Tabla de Contenidos

1. Descripción 5
- **1.1 Versión y entorno** 5
- **1.2 Orden de ejecución** 5
- **1.3 Balance del banco** 5

2. CP-01 Estado de la API 6
- **2.1 Descripción** 6
- **2.2 Condiciones de ejecución** 6
- **2.3 Entrada** 6
- **2.4 Resultado esperado** 6
- **2.5 Evaluación de la Prueba** 6

3. CP-02 Respuesta ante una ruta desconocida 7
- **3.1 Descripción** 7
- **3.2 Condiciones de ejecución** 7
- **3.3 Entrada** 7
- **3.4 Resultado esperado** 7
- **3.5 Evaluación de la Prueba** 7

4. CP-03 Respuesta con la base de datos disponible 8
- **4.1 Descripción** 8
- **4.2 Condiciones de ejecución** 8
- **4.3 Entrada** 8
- **4.4 Resultado esperado** 8
- **4.5 Evaluación de la Prueba** 8

5. CP-04 Caída controlada de MariaDB 9
- **5.1 Descripción** 9
- **5.2 Condiciones de ejecución** 9
- **5.3 Entrada** 9
- **5.4 Resultado esperado** 9
- **5.5 Evaluación de la Prueba** 9

6. CP-05 Inicio de sesión con contraseña incorrecta 10
- **6.1 Descripción** 10
- **6.2 Condiciones de ejecución** 10
- **6.3 Entrada** 10
- **6.4 Resultado esperado** 10
- **6.5 Evaluación de la Prueba** 10

7. CP-06 Filtrado por distrito y por eje 11
- **7.1 Descripción** 11
- **7.2 Condiciones de ejecución** 11
- **7.3 Entrada** 11
- **7.4 Resultado esperado** 11
- **7.5 Evaluación de la Prueba** 11

8. CP-07 Integridad entre vivienda y distrito 12
- **8.1 Descripción** 12
- **8.2 Condiciones de ejecución** 12
- **8.3 Entrada** 12
- **8.4 Resultado esperado** 12
- **8.5 Evaluación de la Prueba** 12

1. Descripción

Este banco comprueba qué partes de UBIKATE se pueden usar en la fase 5. Abarca el acceso de usuarios, los diagnósticos de la API, el filtrado de indicadores por distrito y la integridad de los datos.

1.1 Versión y entorno

Ejecución local del 2 de noviembre de 2026 con Node.js 24.21.0, Express y MariaDB 10.4.32. Se probó el commit ed8b0af de Ubikate-core. La comparación con main (b7c225e) confirmó que src, config y los manifiestos de dependencias son idénticos.

El banco arranca Express en la interfaz local e importa config/ubikate_db.sql en una instancia temporal de MariaDB. Usa una cuenta de aplicación con permisos de lectura y escritura. Los distritos y valores son datos ficticios del ensayo.

1.2 Orden de ejecución

Primero se comprueban la API y la conexión (CP-01 a CP-03). Después se prueban el login, los filtros y la clave foránea (CP-05 a CP-07). CP-04 se ejecuta el último porque detiene la base. El orden de las fichas sigue su identificador.

1.3 Balance del banco

Cinco casos son conformes (CP-01, CP-02, CP-03, CP-04 y CP-07). CP-05 falla porque la clave incorrecta y la vacía se aceptan. CP-06 queda bloqueado porque la ruta propuesta aún no existe. Se observó el 404 y queda pendiente evaluar el filtrado.

Cada ficha incluye una captura del fragmento correspondiente de ejecucion.txt. El registro conserva la hora en UTC. El script banco.cjs permite repetir el ensayo y genera una base nueva en cada ejecución.

2. CP-01 Estado de la API

2.1 Descripción

Comprobar que el servicio se identifica como ubikate-core y publica una marca de tiempo válida. Es la primera comprobación antes de investigar una incidencia de base de datos.

2.2 Condiciones de ejecución

Arrancar Express con src/app.js y anotar el puerto asignado. El caso no necesita que MariaDB responda.

2.3 Entrada

1. Enviar GET /api/health sin cuerpo.

2. Comprobar el estado HTTP y el tipo de contenido. Leer status, service y timestamp y validar que este último se puede convertir a una fecha UTC en formato ISO.

2.4 Resultado esperado

HTTP 200 y JSON con status igual a ok, service igual a ubikate-core y una fecha UTC válida.

2.5 Evaluación de la Prueba

Conforme. Se obtuvo HTTP 200 y el JSON incluyó los tres campos. La fecha de la respuesta fue 2026-11-01T23:50:50.725Z, ya 2 de noviembre en Madrid.

Evidencia · Registro de ejecución de CP-01

```
[CP-01] Estado de la API
GET /api/health
HTTP 200 | application/json; charset=utf-8
{"status":"ok","service":"ubikate-core","timestamp":"2026-11-
01T23:50:50.725Z"}
Estado: Conforme
```

3. CP-02 Respuesta ante una ruta desconocida

3.1 Descripción

Detectar si Express devuelve una página HTML genérica cuando el cliente espera un error de la API. El frontend necesita un cuerpo JSON que pueda interpretar.

3.2 Condiciones de ejecución

Servidor en marcha. /api/qa-inexistente es una ruta de control que no está registrada.

3.3 Entrada

1. Enviar GET /api/qa-inexistente.

2. Leer el código HTTP y Content-Type. Comparar el cuerpo recibido con el mensaje que define el manejador de rutas desconocidas de src/app.js.

3.4 Resultado esperado

HTTP 404, tipo application/json y cuerpo con status igual a error y message igual a «Ruta no encontrada».

3.5 Evaluación de la Prueba

Conforme. La API devolvió 404 y el mensaje previsto en JSON.

Evidencia · Registro de ejecución de CP-02

```
[CP-02] Ruta desconocida
GET /api/qa-inexistente
HTTP 404 | application/json; charset=utf-8
{"status":"error","message":"Ruta no encontrada"}
Estado: Conforme
```

4. CP-03 Respuesta con la base de datos disponible

4.1 Descripción

Verificar que Express llega al motor de datos. La ruta ejecuta SELECT 1 a través del pool de mysql2, de modo que un 200 en este caso sí requiere una conexión a MariaDB.

4.2 Condiciones de ejecución

Instancia local de MariaDB en marcha, esquema del proyecto importado y usuario de prueba con permiso de lectura. La API usa el puerto de esa instancia.

4.3 Entrada

1. Enviar GET /api/health/db.
2. Leer la respuesta JSON y comparar el estado HTTP y el valor de database.

4.4 Resultado esperado

HTTP 200 con status igual a ok y database igual a connected.

4.5 Evaluación de la Prueba

Conforme. Se obtuvo HTTP 200 con database igual a connected. La lectura e integridad de los datos de negocio se comprueban por separado en CP-07.

Evidencia · Registro de ejecución de CP-03

```
[CP-03] Conexión con MariaDB
GET /api/health/db
HTTP 200 | application/json; charset=utf-8
{"status":"ok","database":"connected"}
Estado: Conforme
```

5. CP-04 Caída controlada de MariaDB

5.1 Descripción

Distinguir una caída de MariaDB de una caída completa de la aplicación. Un cliente debe poder saber qué componente sigue disponible.

5.2 Condiciones de ejecución

CP-03 conforme. Mantener Express en marcha y ejecutar este caso al final del banco, una vez terminadas las pruebas que usan la base.

5.3 Entrada

1. Detener únicamente la instancia temporal de MariaDB y esperar a que termine su proceso.

2. Solicitar GET /api/health/db. A continuación, enviar GET /api/health al mismo proceso de Express.

5.4 Resultado esperado

La primera petición devuelve 503, status igual a error, database igual a unreachable y un código de diagnóstico. La segunda devuelve 200 con status igual a ok.

5.5 Evaluación de la Prueba

Conforme. El diagnóstico de base de datos devolvió HTTP 503 y ECONNREFUSED. El endpoint de salud de Express siguió contestando con HTTP 200.

Evidencia · Registro de ejecución de CP-04

```
[CP-04] Caída de la base de datos
MariaDB temporal detenida
GET /api/health/db
HTTP 503 | application/json; charset=utf-8
{"status":"error","database":"unreachable","code":"ECONNREFUSED"}
GET /api/health -> 200 ok
Estado: Conforme
```

6. CP-05 Inicio de sesión con contraseña incorrecta

6.1 Descripción

Verificar que Usuario.login distingue la clave correcta de una incorrecta o vacía. Se comprueba el método del modelo de forma aislada.

6.2 Condiciones de ejecución

Crear en memoria un Usuario con el correo prueba@example.test y una clave conocida. La versión probada no tiene un endpoint de autenticación.

6.3 Entrada

1. Invocar login(correo, clave) con la clave correcta como control.

2. Repetir la llamada con otra clave y después con una cadena vacía. Conservar el mismo usuario en las tres llamadas.

6.4 Resultado esperado

El método devuelve true para la clave correcta y false para la incorrecta y la vacía.

6.5 Evaluación de la Prueba

No conforme. Las tres llamadas devolvieron true. El método acepta cualquier contraseña. Queda pendiente contrastar la clave con el hash almacenado y repetir los tres intentos antes de conectar el login a la interfaz.

Evidencia · Registro de ejecución de CP-05

```
[CP-05] Validación de la contraseña
Usuario.login() | instancia en memoria
Clave correcta -> true
Clave incorrecta -> true
Clave vacía -> true
Estado: No conforme
```

7. CP-06 Filtrado por distrito y por eje

7.1 Descripción

Comprobar que al seleccionar un eje se devuelve solo su indicador para el distrito elegido. Un valor de otra zona o de otro eje cambiaría la lectura del mapa.

7.2 Condiciones de ejecución

Datos ficticios en DATOS_MUNICIPIO. Centro (901) tiene 5, 6, 7 y 8 en seguridad, salud, movilidad y servicios, respectivamente. Retiro (902) tiene 1, 2, 3 y 4 en el mismo orden. Los identificadores son exclusivos del ensayo.

7.3 Entrada

1. Usar el contrato propuesto GET /api/distritos/901/indicadores?eje=seguridad.

2. Repetir para salud, movilidad y servicios. En cada respuesta, comprobar la longitud de la lista, id_distrito, eje y valor.

7.4 Resultado esperado

Cuatro respuestas HTTP 200. Cada lista contiene una sola fila de Centro, con el eje solicitado y su valor de prueba. Ninguna incluye indicadores de Retiro.

7.5 Evaluación de la Prueba

Bloqueado. Las cuatro peticiones devolvieron 404 y «Ruta no encontrada». El prototipo de Figma muestra los selectores, pero esta versión de la API aún no expone el filtro. Hay que acordar la ruta con frontend y repetir las cuatro consultas cuando esté implementada.

Evidencia · Registro de ejecución de CP-06

```
[CP-06] Filtro por distrito y eje
Distritos 901 y 902 | 8 indicadores sintéticos
GET /api/distritos/901/indicadores?eje=…
seguridad -> 404 {"status":"error","message":"Ruta no encontrada"}
salud -> 404 {"status":"error","message":"Ruta no encontrada"}
movilidad -> 404 {"status":"error","message":"Ruta no encontrada"}
servicios -> 404 {"status":"error","message":"Ruta no encontrada"}
Estado: Bloqueado
```

8. CP-07 Integridad entre vivienda y distrito

8.1 Descripción

Comprobar que una vivienda solo se puede guardar si existe su distrito. Probar un alta válida como control permite atribuir el rechazo a la clave foránea.

8.2 Condiciones de ejecución

Mismo esquema InnoDB que usa UBIKATE. Existe el distrito 901 y no existe el 999. No hay viviendas con la dirección ficticia «Vivienda QA». Usar una sola conexión durante la transacción.

8.3 Entrada

1. Abrir una transacción e insertar «Vivienda QA» para el distrito 901. Repetir el alta para el 999.

2. Anotar el código del segundo INSERT y contar las viviendas de prueba. Ejecutar ROLLBACK y repetir el recuento.

8.4 Resultado esperado

El primer INSERT añade una fila. El segundo falla con ER_NO_REFERENCED_ROW_2. Hay una fila antes de ROLLBACK y ninguna después.

8.5 Evaluación de la Prueba

Conforme. MariaDB aceptó la vivienda de Centro y rechazó la referencia al distrito inexistente. El recuento pasó de una fila a cero al deshacer la transacción.

Evidencia · Registro de ejecución de CP-07

```
[CP-07] Integridad de la vivienda
INSERT con distrito 901 -> 1 fila
INSERT con distrito 999 -> ER_NO_REFERENCED_ROW_2
Filas antes de ROLLBACK: 1
Filas después de ROLLBACK: 0
Estado: Conforme
```